

angr-Hardness Toolchain

Installation, Operations & Maintenance Manual

VERSION	1.0
AUTHOR	Stuart Thomas
DATE	2026-04-17
PLATFORM	macOS 12+ (arm64e / x86_64) · Python 3.11 or 3.13
PACKAGE	macos_vuln_toolchain/angr_hardness/
STAGES	C1, C2, C3, C6 operational · C4, C5 killed by design review

Contents

§1	Introduction & System Overview
§2	Installation
§3	Quick Start (5-minute guide)
§4	Operations — C2 RMT Screen
§5	Operations — C3 Template Matching
§6	Operations — C6 Taint Analysis
§7	Operations — C1 Backbone Prioritisation
§8	Interpreting Output
§9	Output File Reference
§10	API Reference
§11	Maintenance
§12	Troubleshooting
§13	Appendix — Known Issues & Workarounds

§1

Introduction & System Overview

Installation, Operations & Maintenance Manual

1.1 Purpose

The angr-Hardness Toolchain is a static binary analysis pipeline for macOS (and optionally Windows) target binaries. It triages compiled binaries — without source code or debug symbols — and identifies functions most likely to harbour memory safety vulnerabilities: out-of-bounds writes, use-after-free conditions, integer overflows flowing to allocators, and type-confusion bugs.

Pipeline stages

Stage	Name	Purpose	Runtime
C1	Backbone Prioritisation	Phase-transition symbolic execution state ranking	32 ms/state avg
C2	RMT Screen	Spectral anomaly detection on call graph	~30 s/binary
C3	Template Matching	Call dataflow pattern matching (5 templates)	~60 s on top-N
C4	TDA	KILLED — homology redundant with cyclomatic M	—
C5	Comp. Sensing	KILLED — RIP fails for coverage bitmaps	—
C6	Taint Analysis	Symbolic taint + PoC input synthesis	mins/function

1.2 Operational Workflow

The standard engagement workflow runs three stages in sequence:

- C2 screen (run_c2_screen.py) — spectral triage of all target binaries, ~30 s per binary. Outputs c2_top_addrs.json with the top-ranked function addresses.
- C3 template scan (run_c3_screen.py) — pattern matching on the C2 top-N functions. Outputs c3_hits.json with template matches and confidence scores.
- C6 targeted symbolic execution — per-function, invoked manually for addresses in c3_hits.json. Outputs c6_alerts.json with taint labels and PoC inputs.

C1 is not a standalone stage. It is an ExplorationTechnique composed inside the C6 symbolic execution loop to prioritise easy paths and defer hard ones.

1.3 Package Structure

macos_vuln_toolchain/		
angr_hardness/		
__init__.py		
exploration_technique.py	# C1: HardnessExplorationTechnique	
semantic_backbone.py	# backbone fraction via Z3 assumptions	
backbone_probe.py	# legacy: pysat Glucose3 path	
dimacs_converter.py	# claripy -> DIMACS CNF	
c2_rmt.py	# C2: C2RMTAnalysis	
Stuart Thomas	c3_templates.py	# C3: C3TemplateAnalysis 2026-04-17
	c6_taint.py	# C6: C6TaintTechnique / C6Analysis
	test_pipeline.py	# unit tests (5/5 passing)
	__init__.py	# C6: C6TaintTechnique / C6Analysis

§2

Installation

Installation, Operations & Maintenance Manual

2.1 Prerequisites

Package	Version	Notes
macOS	12+	Primary platform; arm64e or x86_64
Python	3.11 or 3.13	Avoid 3.12 — angr has known compatibility issues
angr	>= 9.2	Pulls in pyvex, claripy, cle, archinfo, networkx
numpy	recent	Spectral computation
scipy	recent	Eigenvalue solver
z3-solver	recent	Backbone probing via Z3 assumptions
reportlab	optional	PDF generation only
matplotlib	optional	PDF figure generation only
pillow	optional	PDF image embedding only

Do not use Python 3.12

angr has known compatibility issues with Python 3.12 that cause import failures and runtime errors. Use Python 3.11 or Python 3.13.

2.2 Virtual Environment Setup

```
python3.11 -m venv /tmp/angr_venv
source /tmp/angr_venv/bin/activate
pip install angr numpy scipy z3-solver

# Verify installation:
python3 -c "import angr; print(angr.__version__)"
python3 -c "import angr, archinfo, claripy, numpy, scipy, z3; print('OK!)"

# Optional: PDF generation
pip install reportlab matplotlib pillow
```

2.3 Critical: Architecture Override

On macOS, angr defaults to x86_64 when loading universal Mach-O binaries, even on Apple Silicon hardware. This must always be overridden explicitly:

```
import archinfo, angr

proj = angr.Project(
    '/path/to/binary',
    auto_load_libs=False,
    main_opts={'arch': archinfo.arch_from_id('aarch64')}
)

# NEVER pass arch as a bare string
```

§3

Quick Start (5-minute guide)

Installation, Operations & Maintenance Manual

Prerequisites for this guide

Target binary at /usr/libexec/targetd (adjust path as needed)

Virtual environment at /tmp/angr_venv with angr installed

Working directory: macos_vuln_toolchain/

Step 1 — Activate the venv

```
source /tmp/angr_venv/bin/activate
cd /path/to/macos_vuln_toolchain
```

Step 2 — Run the C2 screen (~30 seconds)

Edit the TARGETS list in run_c2_screen.py, then run:

```
python3 run_c2_screen.py
# Produces: c2_results.txt, c2_top_addrs.json
```

Step 3 — Run the C3 template scan (~60 seconds)

```
python3 run_c3_screen.py --top 20
# Reads:      c2_top_addrs.json
# Produces: c3_results.txt, c3_hits.json
```

Step 4 — Inspect the results

```
cat c2_results.txt # anomaly verdict + top-ranked functions
cat c3_results.txt # template matches with confidence scores
cat c3_hits.json   # machine-readable hits for downstream use
```

Step 5 — Targeted symbolic execution (C6)

C6 is invoked per-function. Build a minimal driver targeting the addresses in c3_hits.json. See §6 for full C6 usage details.

```
from angr_hardness.c6_taint import C6TaintTechnique
import archinfo, angr

proj = angr.Project('/usr/libexec/targetd', auto_load_libs=False,
                    main_opts={'arch': archinfo.arch_from_id('aarch64')})
state = proj.factory.call_state(0x10001234) # addr from c3_hits.json
simgr = proj.factory.simgr(state)
simgr.use_technique(C6TaintTechnique())
simgr.run()
findings = simgr.one_deadended.globals.get('c6_findings', [])
for fin in findings: print(f['vuln_class'], f['poc_input'])
```

§4

Operations — C2 RMT Screen

Installation, Operations & Maintenance Manual

4.1 What C2 Does

C2 constructs the call graph of a Mach-O binary, computes three spectral statistics (spectral radius, graph energy, eigenvalue entropy), and compares them against a null distribution derived from 50 configuration-model replicates. Functions are ranked by a combined score weighting eigenvector centrality, cyclomatic complexity, and back-edge count. A binary is flagged ANOMALOUS if the absolute z-score on any of the three statistics exceeds 2.0.

Scoring formula

$$S = 0.4 \times ev + 0.35 \times \log(1+M) + 0.25 \times \log(1+B)$$

Term	Definition	Weight
ev	Eigenvector centrality (0–1)	0.4
M	McCabe cyclomatic complexity = $E - N + 2$	0.35
B	Back-edge count (loop depth indicator)	0.25

4.2 Running C2

Command line

```
python3 run_c2_screen.py
# Edit TARGETS list at the top of run_c2_screen.py to specify binaries
```

Programmatic (from a path)

```
from angr_hardness.c2_rmt import C2RMTAnalysis

result = C2RMTAnalysis('/usr/libexec/targetd').run()
result.print_report()
```

Programmatic (from existing project, avoids double-loading)

```
import archinfo, angr
from angr_hardness.c2_rmt import C2RMTAnalysis

proj = angr.Project('/usr/libexec/targetd', auto_load_libs=False,
                    main_opts={'arch': archinfo.arch_from_id('aarch64')})
result = C2RMTAnalysis.from_project(proj).run()
```

4.3 C2 Result Object

Attribute	Type	Description
result.functions_ranked	list[(int, float)]	Top functions sorted by score, descending
result.anomalous	bool	True if any $ z > 2.0$
result.z_radius	float	Z-score for spectral radius
result.z_energy	float	Z-score for graph energy

§5

Operations — C3 Template Matching

Installation, Operations & Maintenance Manual

5.1 What C3 Does

C3 scans the CFG of a binary looking for source-to-sink call chains that match known vulnerability patterns. Each template specifies source functions (attacker-controlled input entry points), sink functions (dangerous operations), and barrier functions that clear the match if present on the path. A match fires when a source-to-sink path exists in the call graph with no intervening barrier.

5.2 The Five Templates

Template	Source	Sink	Barrier	Conf.
MACH_OOB	mach_msg recv buf field	malloc / calloc size	bounds check	0.82
XPC_TYPE	xpc_dictionary_get_value	typed XPC accessor	xpc_get_type	0.65
INT_OVF	XPC / mach value	arithmetic → allocator	bounds / saturation	0.79
PORT_UAF	mach_port_deallocate	mach port op same name	port epoch check	0.71
IOKIT_OOB	IOConnectCallMethod OOB	memcpy / alloc	bounds check	0.88

5.3 Running C3

Command line

```
python3 run_c3_screen.py          # reads c2_top_addrs.json
python3 run_c3_screen.py --top 20 # limit to top-20 C2 functions
```

Programmatic

```
from angr_hardness.c3_templates import C3TemplateAnalysis
import archinfo, angr

proj = angr.Project('/usr/libexec/targetd', auto_load_libs=False,
                    main_opts={'arch': archinfo.arch_from_id('aarch64')})
c3 = C3TemplateAnalysis(proj)

# Whole binary:
results = c3.run()

# Targeted on C2 top function addresses:
top_addrs = [0x10001234, 0x10002abc]
results = c3.analyse_functions(top_addrs)

for r in results:
    print(r.template_name, r.confidence, r.source_fn, r.sink_fn)
    print('barrier_present:', r.barrier_present)
```

5.4 Adding a New C3 Template

Open `angr_hardness/c3_templates.py` and append a new entry to the `TEMPLATE_BANK` list:

§6

Operations — C6 Taint Analysis

Installation, Operations & Maintenance Manual

6.1 What C6 Does

C6 is a symbolic taint analysis engine built on angr's ExplorationTechnique mechanism. It hooks 18 system functions with SimProcedure stubs that mark return values or output buffers as symbolic bitvectors with labelled taint names. As angr explores paths, claripy's AST tracks those bitvectors through arithmetic operations. When a path reaches a dangerous operation with a tainted operand and a satisfiable constraint system, C6 emits an alert with the taint label, the constraint path, and a solved PoC input.

6.2 Hook Table (18 hooks)

Category	Hooks
Allocation	malloc, calloc, realloc, valloc, mmap
Network / IPC recv	recv, recvfrom, read, mach_msg_recv
XPC	xpc_dictionary_get_value, xpc_array_get_value, xpc_copy
IOKit	IOKit_copyScalar_64, IOKit_copyScalar_32, OSObject_getRef, io_connect_method
Dealloc tracking	free, mach_port_deallocate

6.3 Running C6

C6 is used programmatically. There is no standalone runner script. Build a minimal driver for each target function:

```
from angr_hardness.c6_taint import C6TaintTechnique
from angr_hardness.exploration_technique import HardnessExplorationTechnique
import archinfo, angr

proj = angr.Project('/usr/libexec/targetd', auto_load_libs=False,
                  main_opts={'arch': archinfo.arch_from_id('aarch64')})
state = proj.factory.call_state(0x10001234) # from c3_hits.json
simgr = proj.factory.simgr(state)

# C6 taint technique:
simgr.use_technique(C6TaintTechnique())

# Optional: compose with C1 backbone prioritisation:
simgr.use_technique(HardnessExplorationTechnique(threshold=0.8))

simgr.run()
findings = simgr.one_deadended.globals.get('c6_findings', [])
for f in findings:
    print(f['vuln_class'], f['taint_label'], f['poc_input'])
```

C6 must be targeted, not whole-binary

Running C6 on a full binary without a targeted entry point causes state explosion.

Always scope C6 to a specific function address from c3_hits.json.

When composing with C1, add C6 first so hooks register before C1 begins ranking.

§7

Operations — C1 Backbone Prioritisation

Installation, Operations & Maintenance Manual

7.1 What C1 Does

C1 is an `angr.ExplorationTechnique` that ranks active simulation states by their backbone fraction — the proportion of symbolic variable bits that are forced to a single value by the current path constraints. States with high backbone fraction are near the 3-SAT phase transition (at clause/variable ratio $\alpha \approx 4.267$) and will be slow for the Z3 solver. C1 defers those states to a `hardness_deferred` stash and explores easier states first.

Measured performance (angr crackme benchmark, n=50 symbolic bytes)

Metric	Value
Backbone probing time (avg)	32 ms per state
State reduction	60% fewer states before solution path
Test suite	5/5 tests passing
Spearman ρ (backbone vs CDCL hardness)	+0.43, $p = 0.012$

7.2 C1 Parameters

Parameter	Type	Default	Description
<code>threshold</code>	float	0.8	Backbone fraction cutoff (fixed or adaptive perc...
<code>deferred_stash</code>	str	<code>hardness_deferred</code>	Stash name for deferred states
<code>probe_timeout_s</code>	float	0.05	Z3 timeout per state probe (50 ms)
<code>score_interval</code>	int	1	Score every N exploration steps
<code>min_constraints</code>	int	3	Skip scoring states with fewer constraints
<code>max_score_per_step</code>	int	16	Cap number of states scored per step
<code>adaptive_threshold</code>	bool	True	Defer top 20% hardest (percentile mode)
<code>log_file</code>	str None	None	CSV log path for per-state backbone scores

7.3 Using C1

```

from angr_hardness.exploration_technique import HardnessExplorationTechnique
import archinfo, angr

proj = angr.Project('/path/to/binary', auto_load_libs=False,
                    main_opts={'arch': archinfo.arch_from_id('aarch64')})
state = proj.factory.entry_state()
simgr = proj.factory.simgr(state)

simgr.use_technique(HardnessExplorationTechnique(
    threshold=0.8,
    adaptive_threshold=True,
    probe_timeout_s=0.05,
    log_file='/tmp/hardness_log.csv',

```

§8

Interpreting Output

Installation, Operations & Maintenance Manual

8.1 C2 Anomaly Verdicts

A C2 ANOMALOUS verdict means the call graph structure deviates significantly from what would be expected from a random graph with the same degree sequence. This is a triage signal, not a confirmation of vulnerability.

Pattern	Likely meaning
z_energy strongly negative	Massively structured internal complexity — many high-M functions
z_radius strongly positive	Unusually strong cyclic structure — persistent loops
z_entropy strongly positive	Noise-dominated spectrum — may be Swift/ObjC vtable noise (FP)
z_entropy strongly negative	Rigid hierarchy — monolithic processing without abstraction

Reference values from audit corpus

Binary	z_radius	z_energy	z_entropy	Verdict
smbd	-3.56	-3.01	-0.30	ANOMALOUS
opendirectoryd	-2.50	-1.88	-1.10	ANOMALOUS
mDNSResponder	-1.85	-22.33	-142.37	ANOMALOUS
amfid	-0.10	-0.08	-0.05	NORMAL

8.2 C3 Confidence Scores

Confidence	Level	Interpretation
≥ 0.85	High	Source-to-sink path very likely unmitigated
0.70–0.84	Medium	Worth manual review of the call chain
< 0.70	Low	Barrier may exist; verify call chain manually

A C3 match with barrier_present=True is not a vulnerability finding. A barrier function was found on the source-to-sink path. Record and move on.

8.3 C6 Alerts

Field	Meaning
vuln_class	OOB_WRITE OOB_READ UAF INT_OVF TYPE_CONFUSION
taint_label	Hook name that introduced the taint (e.g. recv_buf_3)
constraint	The SMT constraint path that led to the vulnerable state
poc_input	Concrete hex byte sequence (Z3-solved); valid attacker input

§9

Output File Reference

Installation, Operations & Maintenance Manual

9.1 File Summary

File	Format	Contents
c2_results.txt	Plain text	C2 report: anomaly verdict, z-scores, top-20 functions
c2_top_addrs.json	JSON	{binary_label: [{addr, score, cyclomatic, back_edges, name}]}
c3_results.txt	Plain text	C3 report: template hits with confidence and function names
c3_hits.json	JSON	{binary_label: [{addr, template, confidence, source_fn, sink_fn}]}
c6_alerts.json	JSON	[{addr, vuln_class, taint_label, constraint, poc_input}]
hardness_log.csv	CSV	Per-state backbone scores (only if log_file set in C1)

9.2 c2_top_addrs.json Schema

```
{
  "targetd": [
    {
      "addr": 4295012928,
      "score": 2.341,
      "cyclomatic": 155,
      "back_edges": 12,
      "name": "sub_100012340"
    }
  ]
}
```

9.3 c3_hits.json Schema

```
{
  "targetd": [
    {
      "addr": 4295012928,
      "template": "MACH_00B",
      "confidence": 0.82,
      "source_fn": "mach_msg",
      "sink_fn": "malloc"
    }
  ]
}
```

9.4 c6_alerts.json Schema

```
{
  "addr": 4295012928,
  "vuln_class": "MACH_00B",
  "taint_label": "MACH_00B",
  "constraint": "MACH_00B",
  "poc_input": "MACH_00B"
```

§10

API Reference

Installation, Operations & Maintenance Manual

10.1 C1 — HardnessExplorationTechnique

```
# angr_hardness.exploration_technique.HardnessExplorationTechnique

HardnessExplorationTechnique(
    threshold=0.8,
    deferred_stash='hardness_deferred',
    probe_timeout_s=0.05,
    score_interval=1,
    min_constraints=3,
    max_score_per_step=16,
    adaptive_threshold=True,
    log_file=None,
)

# Methods called by angr automatically:
# .setup(simgr)          -- called on attachment
# .step(simgr, **kwargs) -- scores and defers states each step
```

10.2 C2 — C2RMTAnalysis

```
# angr_hardness.c2_rmt.C2RMTAnalysis

C2RMTAnalysis(binary_path: str)
C2RMTAnalysis.from_project(proj: angr.Project) # class method

.run() -> C2Result

# C2Result attributes:
.functions_ranked  list[(int, float)] # (addr, score), sorted desc
.anomalous         bool
.z_radius          float
.z_energy          float
.z_entropy         float
.print_report()    -> None # human-readable to stdout
```

10.3 C3 — C3TemplateAnalysis

```
# angr_hardness.c3_templates.C3TemplateAnalysis

C3TemplateAnalysis(proj: angr.Project)

.run() -> list[C3Match]
.analyse_functions(addr: list[int]) -> list[C3Match]
```

```
# C3Match attributes:
```

```
Stuart Thomas
.template_name  str
.confidence     float
.source_fn      str
```

§11

Maintenance

Installation, Operations & Maintenance Manual

11.1 Update Schedule

Frequency	Action	Notes
Weekly	<code>pip install --upgrade angr</code>	angr releases frequently; check changelog for CLE/pyvex re...
Before each engagement	Run test suite	<code>python3 -m pytest angr_hardness/test_pipeline.py -v</code>
After angr upgrade	Run validation scripts	<code>python3 angr_hardness/validate_c3.py && python3 angr_har...</code>
Before major changes	Backup	<code>zip -r angr_hardness_backup_\$(date +%Y%m%d).zip angr_...</code>

angr API stability

angr upgrades frequently change internal APIs in pyvex, cle, and claripy.

After any pip upgrade angr, always run the full validation suite before

using the toolchain on a live engagement.

Silent regressions in hook resolution produce false negatives with no error output.

11.2 Test Suite Commands

```
# Full unit tests (must pass 5/5):
python3 -m pytest angr_hardness/test_pipeline.py -v

# C3 regression validation:
python3 angr_hardness/validate_c3.py

# C6 regression validation:
python3 angr_hardness/validate_c6.py

# Check angr version:
python3 -c "import angr; print(angr.__version__)"

# Verify all imports:
python3 -c "import angr, archinfo, claripy, numpy, scipy, z3; print('OK')"
```

11.3 Adding a New Binary Target

Edit the TARGETS list in `run_c2_screen.py`. Before adding a binary, verify CLE can load it cleanly:

```
import archinfo, angr
proj = angr.Project('/your/binary', auto_load_libs=False,
                    main_opts={'arch': archinfo.arch_from_id('aarch64')})
n = len(list(proj.kb.functions.values()))
print(f'{n} functions found')

# If n == 0: likely arm64e PAC CLE bug (see §13 Issue 2)
```

§12

Troubleshooting

Installation, Operations & Maintenance Manual

12.1 Error Messages and Fixes

SimEngineError: CLE could not find any executable sections

Cause: Binary is encrypted, packed, or uses an unrecognised segment layout.

Fix: Use `otool` / Ghidra for manual analysis. Skip in batch runs.

KeyError: 'Win32' in SYSCALL_CC

Cause: Windows ARM64 binary loaded without the SYSCALL_CC monkey-patch.

Fix: Apply Windows ARM64 patch before any `Project()` construction (see §13 Issue 4).

AngrMemoryError: Not enough memory

Cause: Binary exceeds ~3.3 MB — CFGFast OOMs on 16 GB RAM hardware.

Fix: Use sparse eigenvalue mode or skip. Subset CFG to region of interest.

archinfo error: arch must be an archinfo.Arch instance

Cause: arch passed as a bare string to `main_opts`.

Fix: Use `archinfo.arch_from_id('aarch64')` — always an Arch object, never a string.

claripy.errors.BackendError: Cannot convert

Cause: Constraint system too complex for Z3 within timeout.

Fix: Reduce `max_score_per_step` in C1; narrow the entry state scope.

AttributeError: 'NoneType' has no attribute 'variables'

Cause: C6 hook returned None instead of a BVS.

Fix: Check hook's `run()` method — must always return a BVS, never None.

C3 returns empty results

Cause: CFGFast missed the function (ObjC dispatch / Swift vttables).

Fix: Use `analyse_functions()` with a manual address list from `nm` or `otool`.

z-scores all near 0

Cause: Call graph has fewer than 100 nodes.

Fix: Low-confidence result. Use function-level metrics (M, back-edges, ev) only.

All states immediately deferred by C1

Cause: All paths are hard (backbone fraction > threshold).

Fix: Lower threshold to 0.6 or set `adaptive_threshold=True` (percentile mode).

§13

Appendix — Known Issues & Workarounds

Installation, Operations & Maintenance Manual

Issue 1: angr CFG Size Limit (~3.3 MB Mach-O)

angr's CFGFast analysis will exhaust available memory on binaries larger than approximately 3.3 MB when running on a 16 GB RAM machine. This is a practical ceiling, not a hard limit; it varies with binary complexity. For large targets, subset the analysis to a region of interest by providing a list of function addresses to `C3TemplateAnalysis.analyse_functions()` rather than calling `run()` on the full binary.

Issue 2: arm64e PAC / Chained Fixup Relocations (airportd, biometrickitd)

CLE has a known bug handling binaries that use arm64e pointer authentication codes with chained fixup relocations. Affected binaries include `airportd` and `biometrickitd`. The symptom is an empty function list after CFGFast. Workaround: extract function addresses via `nm` or `otool` and pass them directly to `analyse_functions()`:

```
# Extract addresses:
nm -n /System/Library/.../biometrickitd | grep ' T ' | awk '{print $1}' > addrs.txt

# Use in C3:
addrs = [int(x, 16) for x in open('addrs.txt').read().splitlines()]
results = c3.analyse_functions(addrs)
```

Issue 3: Python 3.12 Compatibility

Do not use Python 3.12

angr has known compatibility issues with Python 3.12 that cause import failures.

Use Python 3.11 or Python 3.13. Do not attempt to diagnose 3.12 failures —

they are upstream angr issues not fixable in this codebase.

Issue 4: Windows ARM64 SYSCALL_CC Patch

When analysing Windows ARM64 PE binaries, angr's calling convention registry is missing the Win32 entry for AARCH64. This must be patched before any `Project()` construction. Additionally, use `pefile` + `capstone` for IAT resolution rather than the unreliable angr KB function lookup:

```
# Apply before ANY Project() construction:
from angr.calling_conventions import SYSCALL_CC
SYSCALL_CC['AARCH64']['Win32'] = SYSCALL_CC['AARCH64'].get('Linux')

# IAT resolution (replaces unreliable angr KB for Windows PE):
import pefile, capstone
pe = pefile.PE('target.exe')
iat = {
    imp.address: imp.name.decode()
    for entry in pe.DIRECTORY_ENTRY_IMPORT
    for imp in entry.imports if imp.name
}
```

Issue 5: VEX IR Constant-Folding (ARM64 Compiler Optimisation)